

فاز ۱۳: Spring Boot – خلاصه و کاربردی

این بخش برای مصاحبه‌هایی که ازت Spring Boot می‌پرسن ضروریه. نیازی نیست همه جزئیات رو بدونی، ولی مفاهیم کلیدی رو باید بلد باشی.

۱۳.۱ چرخه حیات Bean ها در Spring

Bean: یه شیء که توسط Spring Container مدیریت میشه.

مراحل چرخه حیات:

1. Instantiation: سازنده صدا زده میشه

2. Dependency Injection: وابستگی‌ها تزریق میشن

3. Initialization: متدهای @PostConstruct و afterPropertiesSet اجرا میشن

4. Bean: Ready for Use آماده استفاده است

5. Destruction: متدهای @PreDestroy اجرا میشن

```

1 @Component
2 class MyBean {
3     public MyBean() {
4         System.out.println("1. سازنده");
5     }
6
7     @Autowired
8     public void setDependency(SomeService service) {
9         System.out.println("2. تزریق وابستگی");
10    }

```

```
11
12 @PostConstruct
13 public void init() {
14     System.out.println("مقداردهی اولیه 3.");
15 }
16
17 @PreDestroy
18 public void destroy() {
19     System.out.println("پاک سازی 4.");
20 }
21 }
```

۱۳.۲ انواع تزریق وابستگی (Dependency Injection)

۱. Constructor Injection (توصیه شده ✓)

```
1 @Service
2 class UserService {
3     private final UserRepository repository;
4
5     public UserService(UserRepository repository) {
6         this.repository = repository;
7     }
8 }
```

۲. Setter Injection

```

1 @Service
2 class UserService {
3     private UserRepository repository;
4
5     @Autowired
6     public void setRepository(UserRepository repository) {
7         this.repository = repository;
8     }
9 }

```

۳. Field Injection (کمتر توصیه میشه)

```

1 @Service
2 class UserService {
3     @Autowired
4     private UserRepository repository;
5 }

```

۱۳.۳ Transactional@ چطور کار می‌کنه؟

Spring با Proxy دور کلاس می‌پیچه و قبل و بعد از متد، Transaction رو مدیریت می‌کنه.

```

1 @Service
2 class OrderService {
3     @Autowired

```

```

4 private OrderRepository orderRepository;
5
6 @Transactional
7 public void createOrder(Order order) {
8     // ۱. Transaction شروع همیشه
9     orderRepository.save(order);
10    // ۲. Transaction Commit (اگر خطا نبود) همیشه
11    // ۳. Rollback، اگر خطا بود همیشه
12 }
13 }

```

نکات مهم:

• @Transactional روی متدهای public کار می‌کند (به خاطر Proxy)

• روی متدهای private کار نمی‌کند

• RuntimeException باعث Rollback همیشه، Checked Exception نه (مگر اینکه تنظیم کنی)

```

1 // Exception برای همه Rollback
2 @Transactional(rollbackFor = Exception.class)
3 public void save() { ... }
4
5 // فقط RuntimeException برای
6 @Transactional
7 public void save() { ... }

```

۱۳.۴ Repository@ و Service@ و RestController@

Annotation	کاربرد
RestController@	کنترلر REST API (ترکیب Controller + @ResponseBody@)
Service@	لایه Business Logic
Repository@	لایه دسترسی به دیتابیس (با ترجمه خودکار Exception)
Component@	عمومی‌ترین، برای هر Bean دلخواه

```

1 @RestController
2 @RequestMapping("/api/users")
3 class UserController {
4     @Autowired
5     private UserService userService;
6
7     @GetMapping("/{id}")
8     public User getUser(@PathVariable Long id) {
9         return userService.findById(id);
10    }
11 }
12
13 @Service
14 class UserService {
15     @Autowired
16     private UserRepository repository;
17
18     public User findById(Long id) {
19         return repository.findById(id).orElseThrow();

```

```
20 }  
21 }  
22  
23 @Repository  
24 interface UserRepository extends JpaRepository<User, Long> {  
25 }
```

۱۳.۵ application.properties مهم ترین تنظیمات

```
1 # properties config  
2 # سرور  
3 server.port=8080  
4 server.servlet.context-path=/api  
5 # دیتابیس  
6 spring.datasource.url=jdbc:mysql://localhost:3306/mydb  
7 spring.datasource.username=root  
8 spring.datasource.password=1234  
9 # JPA  
10 spring.jpa.hibernate.ddl-auto=update  
11 spring.jpa.show-sql=true  
12 spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQL8Dialect  
13 # لاگ  
14 logging.level.org.springframework=INFO  
15 logging.level.com.myapp=DEBUG
```

۱۳.۶ Scope های Bean در Spring

Scope	توضیح
singleton (پیش فرض)	فقط یک نمونه در کل برنامه
prototype	هر بار که درخواست بشه، یه نمونه جدید
request	برای هر درخواست HTTP، یه نمونه (فقط در Web)
session	برای هر نشست کاربر، یه نمونه (فقط در Web)

```

1 @Component
2 @Scope("prototype")
3 class MyPrototypeBean { ... }

```

۳. Spring Bean ها (مفصل تر)

۱۳.۷ Bean های Spring – کامل

۱۳.۷.۱ چرخه حیات کامل Bean

```

1 |
2 |
3 | 1. Instantiation (ساخت شیء)
4 |   ↓
5 | 2. Populate Properties (تزریق وابستگی ها)
6 |   ↓

```

```

7 | 3. BeanNameAware (پیا‌ده‌سازی شده اگر setBeanName)
8 |   ↓
9 | 4. BeanClassLoaderAware (پیا‌ده‌سازی شده اگر setBeanClassLoader)
10 |   ↓
11 | 5. BeanFactoryAware (پیا‌ده‌سازی شده اگر setBeanFactory)
12 |   ↓
13 | 6. @PostConstruct (متد init)
14 |   ↓
15 | 7. InitializingBean (پیا‌ده‌سازی شده اگر afterPropertiesSet)
16 |   ↓
17 | 8. Bean Ready! (آماده استفاده)
18 |   ↓
19 | 9. @PreDestroy (متد destroy)
20 |   ↓
21 | 10. DisposableBean (پیا‌ده‌سازی شده اگر destroy)
22 |
23 |

```

۱۳.۷.۲ انواع Scopes

```

1 // ۱. Singleton (پیش‌فرض) - یک نمونه برای کل برنامه
2 @Component
3 @Scope("singleton")
4 class SingletonBean { ... }
5
6 // ۲. Prototype - هر بار یک نمونه جدید
7 @Component
8 @Scope("prototype")

```


13-Java-Spring-Boot

```
9 class PrototypeBean { ... }
10
11 // ۳. Request - برای هر درخواست HTTP
12 @Component
13 @Scope("request")
14 class RequestBean { ... }
15
16 // ۴. Session - برای هر نشست کاربر
17 @Component
18 @Scope("session")
19 class SessionBean { ... }
20 // ۵. Application - یک نمونه برای کل ServletContext
21 @Component
22 @Scope("application")
23 class ApplicationBean { ... }
```

۱۳.۷.۳ Bean Definition (روش‌های تعریف Bean)

```
1 // a. با @Component
2 @Component
3 class MyComponent { ... }
4
5 // b. با @Service
6 @Service
7 class MyService { ... }
8
9 // c. با @Repository
10 @Repository
```

```

11 class MyRepository { ... }
12
13 // d. با @Controller / @RestController
14 @RestController
15 class MyController { ... }
16
17 // e. در کلاس @Configuration با @Bean
18 @Configuration
19 class AppConfig {
20     @Bean
21     public MyBean myBean() {
22         return new MyBean();
23     }
24
25     @Bean
26     public MyBean myBeanWithParams(AnotherBean another) {
27         return new MyBean(another);
28     }
29 }
30
31 // f. با @Import
32 @Import({MyConfig.class, AnotherConfig.class})
33 class MainConfig { ... }

```

34 ۱۳.۷.۴ Bean Wiring (ها Bean اتصال)

35 // ۱. Constructor Injection (توصیه شده)

```

36 @Service
37 class UserService {
38     private final UserRepository repository;
39
40     // نیست @Autowired اگر فقط یک سازنده باشد، نیازی به
41     public UserService(UserRepository repository) {
42         this.repository = repository;
43     }
44 }
45
46 // ۲. Setter Injection
47 @Service
48 class UserService {
49     private UserRepository repository;
50
51     @Autowired
52     public void setRepository(UserRepository repository) {
53         this.repository = repository;
54     }
55 }
56
57 // ۳. Field Injection
58 @Service
59 class UserService {
60     @Autowired
61     private UserRepository repository;

```

```

62 }
63
64 // ۴. @Qualifier (از یه نوع هستن Bean وقتی چند تا)
65 @Service
66 class ReportService {
67     @Autowired
68     @Qualifier("pdfReport")
69     private ReportGenerator pdfGenerator;
70
71     @Autowired
72     @Qualifier("excelReport")
73     private ReportGenerator excelGenerator;
74 }
75
76 // ۵. @Primary (Bean اولویت با این)
77 @Configuration
78 class ReportConfig {
79     @Bean
80     @Primary
81     public ReportGenerator pdfGenerator() {
82         return new PdfGenerator();
83     }
84
85     @Bean
86     public ReportGenerator excelGenerator() {
87         return new ExcelGenerator();

```

```

88 }
89 }
90
91 // ۶. @Lazy (ساخت دیر هنگام)
92 @Service
93 class UserService {
94     @Autowired
95     @Lazy
96     private HeavyService heavyService; // فقط وقتی استفاده بشه ساخته میشه
97 }

```

۱۳.۷.۵ Conditional Beans (ساخت شرطی)

```

1 @Configuration
2 class ConditionalConfig {
3
4     // ۱. اگه کلاس مشخصی در Classpath بود
5     @ConditionalOnClass(Driver.class)
6     @Bean
7     public DataSource mysqlDataSource() { ... }
8
9     // ۲. اگه کلاس مشخصی در Classpath نبود
10    @ConditionalOnMissingClass("com.mysql.jdbc.Driver")
11    @Bean
12    public DataSource h2DataSource() { ... }
13
14    // ۳. اگه پراپرتی مشخصی تنظیم شده بود

```

```
15 @ConditionalOnProperty(name = "app.use.mysql", havingValue = "true")
16 @Bean
17 public DatabaseService mysqlService() { ... }
18
19 // ۴. خاصی موجود بود Bean اگه
20 @ConditionalOnBean(MyService.class)
21 @Bean
22 public AnotherService anotherService() { ... }
23
24 // ۵. ترکیب شرطها
25 @ConditionalOnProperty(name = "app.cache.enabled")
26 @ConditionalOnBean(CacheManager.class)
27 @Bean
28 public CacheService cacheService() { ... }
29 }
```